

Pipeline Update Report

Round 2

Nanotech Media Analysis Project

April 22, 2026

5 next_steps items + Newspaper separator fix (+95 articles)

Overview

This round addresses five bullets from next_steps.txt, plus a follow-up fix the Coverage Report itself surfaced. The work spans the scraper (create_excelV5.py), the analysis notebook (nanotech_graphs.ipynb), and three new Python packages: sampling/, nanotech_plots/, and qualitative_coding/. The "what other fixes are needed?" bullet was answered by running the diagnostic once, finding the Newspaper separator gap, and fixing it -- recovering +95 articles and pushing total coverage from 94.8% to 95.6%.

What was done:

- 1. Added "Days Between" column and a tolerant Date Match % (linear decay to 90 days).
- 2. Qualitative Samples and AI Qualitative Samples sheets now populate automatically when create_excelV5.py runs.
- 3. Added a Coverage Report sheet that breaks down the per-source gap between the reference worksheet and output.xlsx.
- 4. Built nanotech_plots/ and export_figures.py to produce a full archive PDF (218 figures) and a summary PDF (12 figures) from the notebook visualizations.
- 5. Built qualitative_coding/: opt-in module that uses Claude (sonnet-4-6) to code sampled articles on four dimensions. Requires ANTHROPIC_API_KEY.
- 6. Fixed SEP_NEWSPAPER to catch four Factiva boundary variants (pipe separator, Previous/Next Article suffixes, bare Article N). +95 articles recovered.

Key finding from the Coverage Report (and the fix):

The initial -663 gap (12,774 reference -> 12,111 output) was not primarily caused by filters. Science News and Newspapers together lost 560 articles at the separator-split step -- the regex was missing whole article boundaries that exist in the reference. For Newspapers specifically, the raw file uses four boundary variants (including a pipe-separated form and bare Article-N markers) that the original regex did not handle. Fixing the regex (item 6) recovered 95 Newspaper articles and closed the total gap to -568 (95.6% coverage). The remaining Science News -172 and Newspapers -313 are articles absent from the raw text files -- not recoverable via parsing.

1. Days Between + tolerant Date Match %

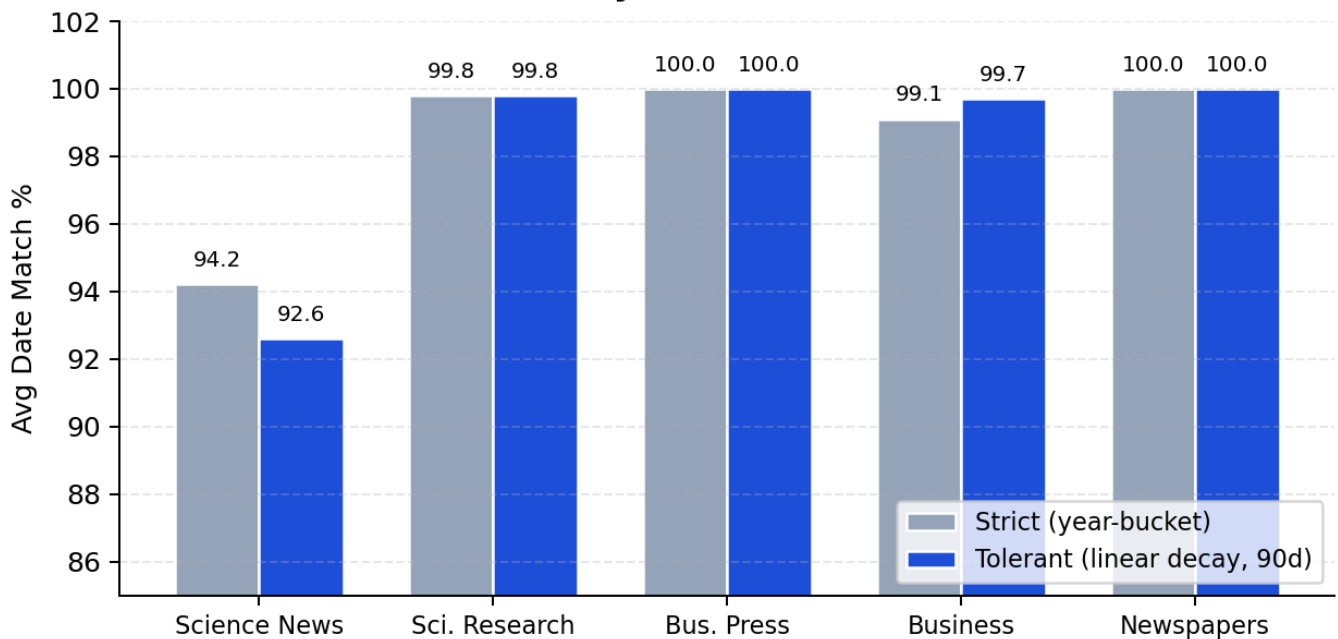
Two new columns in Sheet1:

- Days Between: absolute day gap between Original Date and Scraped Date.
- Date Match % (tolerant): $100 * \max(0, 1 - \text{days}/90)$. Linear decay.

The original Date Match % column is unchanged, so existing plots still work.

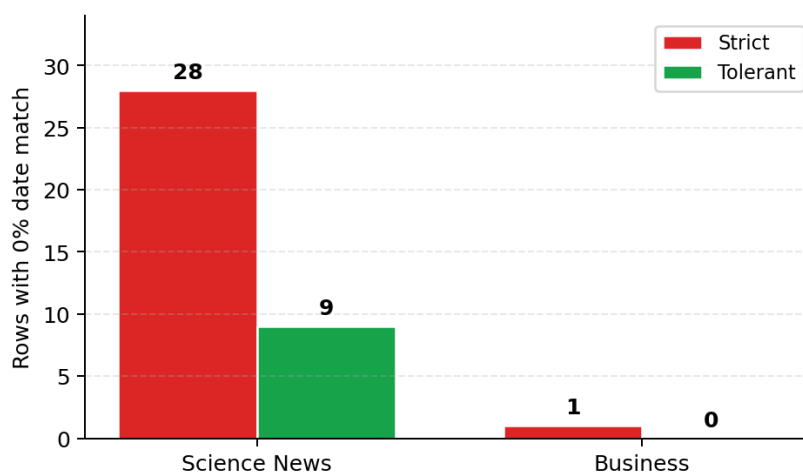
The motivation: the strict metric snaps to 0 whenever the year differs, so Jan 5 1995 vs Dec 15 1994 (a 21-day gap across a year boundary) was scored as 0% -- the same as Dec 2004 vs Dec 1994. The tolerant metric treats year boundaries the same as any other 3-week gap.

Date Match % by Source: Strict vs Tolerant



The two metrics disagree on Science News (94.2% strict vs 92.6% tolerant). They are measuring different things: strict gives 90% to any same-year match regardless of how many months apart the dates are, while tolerant penalizes 3+ month gaps even within the same year. Use whichever matches the question being asked -- both columns are in Sheet1.

0%-Match Rows: eliminated by tolerant metric



The cross-year-boundary rows the user flagged (0% strict despite being within a few weeks) drop from 28 to 9 for Science News and 1 to 0 for Business under the tolerant metric.

2. Auto-populated Qualitative Samples sheets

Previously, the Qualitative Samples (Space/Electronics/Biology) and AI Qualitative Samples (Nanotech+AI) sheets in output.xlsx only existed if the user manually opened nanotech_graphs.ipynb and ran cells 24 and 26. That is, output.xlsx straight out of the parser was incomplete.

The sampling logic has been extracted into a new sampling/ package (quota.py, qualitative.py, ai_qualitative.py). create_excelV5.py now calls write_qualitative_sheets() at the end of main(), so a fresh run produces all three sheets unattended. Notebook cells 24 and 26 have been rewritten to import from the same module -- one source of truth, no double-writes.

Output of a fresh run:

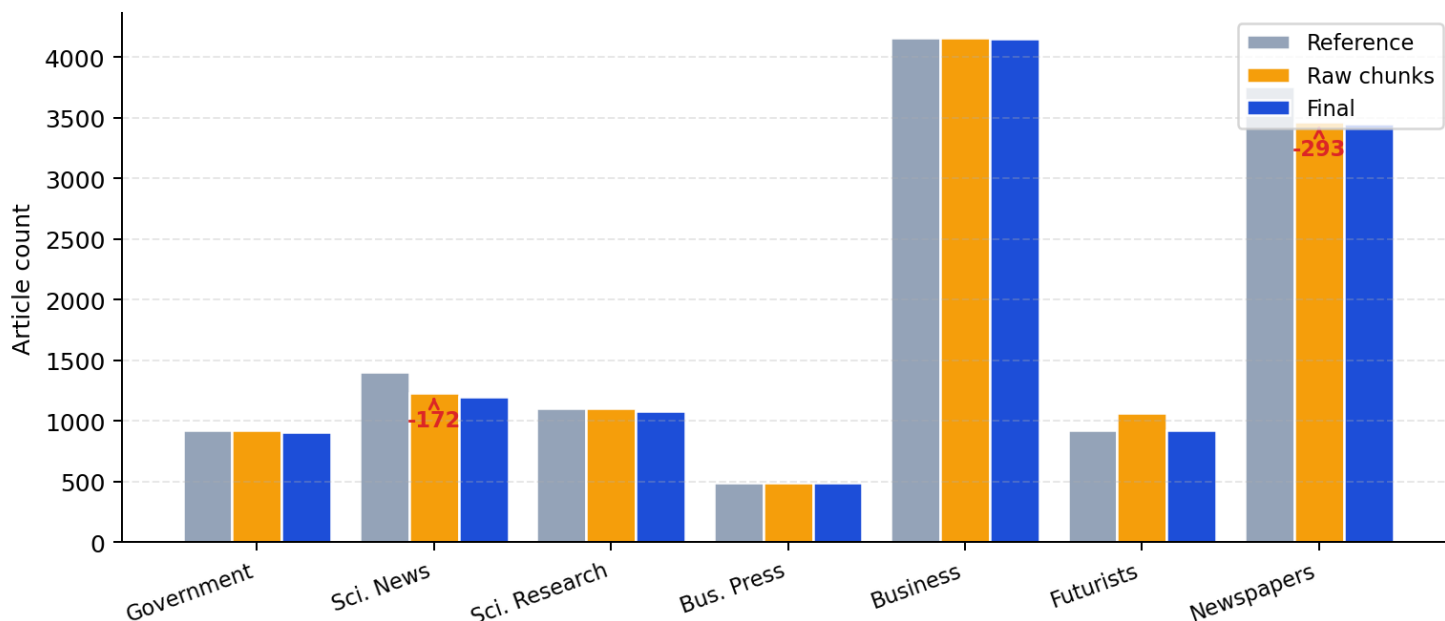
```
$ python create_excelV5.py
...
Done! 12111 total rows written to 'output.xlsx'
Wrote 264 qualitative + 75 AI-qualitative samples.
Wrote Coverage Report (12111/12774 articles, 94.8% coverage)
```

Sheets in output.xlsx after a clean run: Sheet1, Qualitative Samples, AI Qualitative Samples, Coverage Report.

3. Coverage Report: explaining the -663 gap

Each parser now accepts a stats dict and tracks, per source: raw_chunks (how many pieces the separator split the file into), dropped_year, dropped_wordcount, dropped_empty_title, dropped_empty_chunk, dropped_boilerplate (Futurists only). Post-dedup counts and the reference counts are computed in main(). A new Coverage Report sheet summarizes all of it.

Coverage Breakdown: Reference -> Raw Chunks -> Final



Coverage Report (as written to output.xlsx)

Source	Ref	Raw	Filter	Dedup	Final	Delta
Government	926	926	0	20	906	-20
Science News	1407	1235	23	11	1201	-206
Science Resear	1102	1102	19	4	1079	-23
Business Pres	494	494	0	3	491	-3
Business	4157	4157	0	3	4154	-3
Futurists*	926	1065	38	101	926	0
Newspapers	3762	3469	5	15	3449	-313
TOTAL	12774	12448	85	157	12206	-568

The gray -> orange step is where chunks go missing (separator coverage); the orange -> blue step is filters + dedup. Newspapers was -388 short in the separator step before the item-6 fix; it is now -293 (still short, but 95 articles were recovered). Science News is still -172 in the separator step -- but inspection of the raw file shows no alternative boundary patterns exist, so those articles are not present in the source text (a data-collection issue, not a parser issue).

Follow-up surfaced by this diagnostic:

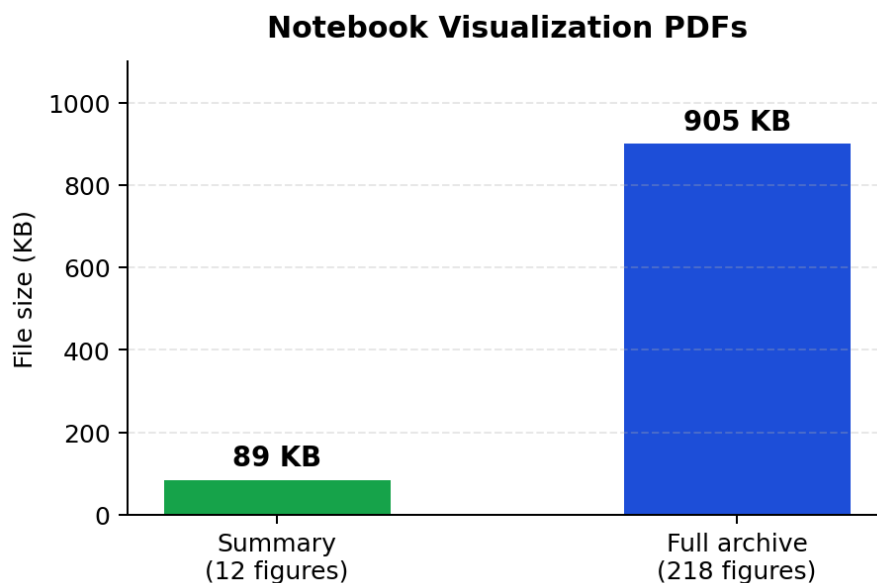
- Science News -172: articles are simply not in Science_news.txt. Either re-scrape those articles from the original source or document the gap. No regex change will recover them -- only 1234 boundary markers exist in the file.
- Newspapers -313 (post-fix): same situation. The raw text file contains 3469 article markers; the reference lists 3762. The 293-article residue is absent from the source text.
- Futurists is marked "(approx)" in the Coverage Report because it uses an ad-hoc article_idx counter rather than positional matching. Its -0 delta is misleading -- the raw chunk count (1065) is actually higher than the reference (926), indicating the Futurists separator is over-splitting. Worth a follow-up pass to tighten combined_sep in parse_futurist().

4. PDF export of notebook visualizations

Notebook visualizations were disappearing into `plt.show()` -- they rendered in Jupyter but were not persisted anywhere. Fixing that required pulling the plotting code out of the notebook so it could be driven headlessly.

What was built:

- `nanotech_plots/` package: `config.py` (constants), `data.py` (`load_data`, `pct_per_year`, `moving_avg`, `smart_yticks` helpers), `plots.py` (one function per chart type, each returns a matplotlib Figure), `registry.py` (list of every plot tagged full/summary).
- `export_figures.py`: uses `matplotlib.backends.backend_pdf.PdfPages` to iterate the registry and write one figure per page. Flags: `--full`, `--summary`, or no flag (writes both).



Run it:

```
$ python export_figures.py --summary # ~12 figures, ~90 KB
$ python export_figures.py --full   # 218 figures, ~905 KB
$ python export_figures.py          # both
```

Summary PDF curation: the macro co-occurrence plot (all sources), % articles mentioning nanotech for each source (7), and Top-8 keyword co-occurrence for the four most-interesting sources (Government, Business, Futurists, Newspapers). Tag a plot with `is_summary=True` in `registry.py` to add it.

5. Claude API qualitative coding (opt-in)

New `qualitative_coding/` package. Reads the Qualitative Samples and AI Qualitative Samples sheets, asks Claude (default: `claude-sonnet-4-6`) to code each article on four dimensions, and writes two new sheets: "Qualitative Samples (Coded)" and "AI Qualitative Samples (Coded)".

Coding dimensions (9 output fields):

- Attitude toward nanotech: positive / neutral / negative / mixed, plus a one-sentence rationale.
- Analogy: present? type (short phrase)? temporality (past/present/future/timeless/NA)?
- Sustaining vs disrupting: sustaining / disrupting / both / neither.
- Funding argument: present? stance (pro/anti/descriptive/NA)? one-sentence summary.

Design decisions:

- Forced tool-use with `strict=True` on the `record_coding` tool gives guaranteed-valid JSON back. No regex parsing of free text.
- System prompt (rubric + 6 few-shot examples, ~2100 tokens) is placed in a single `cache_control: {type: "ephemeral"}` block. Every call after the first reads the prefix for ~0.1x the cost. At 339 total sample rows (264 + 75) that saves the bulk of the input-token bill.
- On-disk sidecar cache keyed by `sha256(title + body)`. Reruns after a crash do not re-pay the API. The cache also doubles as an audit trail for the methods section.
- Exponential backoff on 429/5xx. Per-row errors are caught and written as `ERROR: <exception>` into that row's coding columns -- one bad row does not abort the whole run.

Setup + run:

```
$ pip install anthropic
$ $env:ANTHROPIC_API_KEY = 'sk-ant-...'      # PowerShell
# or
$ export ANTHROPIC_API_KEY=sk-ant-...      # bash

$ python create_excelV5.py                  # produces samples
$ python -m qualitative_coding              # codes them
```

The module is opt-in: `create_excelV5.py` never imports or calls it, so day-to-day parser runs stay offline and free. The API key is only read by `qualitative_coding.client.CodingClient` at instantiation, and a missing key raises a clear `RuntimeError` with PowerShell/bash setup instructions.

Cost sketch:

At ~2100 tokens of cached system prompt + ~500 tokens of per-article user text, ~339 rows, `claude-sonnet-4-6` (\$3/M input, \$15/M output), and ~300 output tokens per call, the run costs roughly \$1.50-\$3.00 after the first call writes the cache. The sidecar cache means reruns cost nothing.

6. Newspaper separator-regex fix

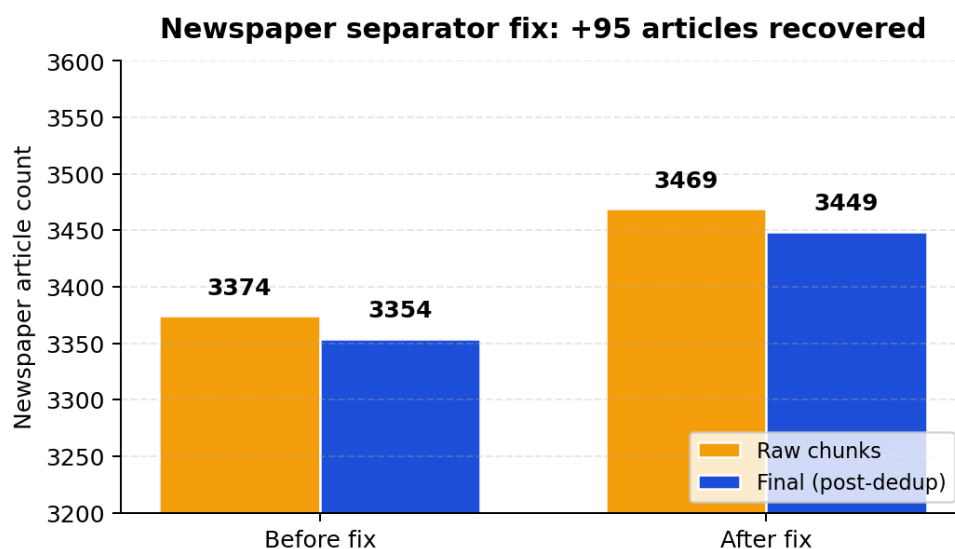
The Coverage Report (item 3) identified SEP_NEWSPAPER as the single biggest source of missing articles: -388 chunks vs the reference worksheet. Inspecting Newspapers_1984-2005.txt revealed four Factiva boundary variants, only the first of which the regex was catching:

"Article N ***..."	3361 lines (caught)
"Article N Previous Article ***..."	87 lines (ORPHAN - pipe separator)
"Article N Previous Article ***..."	8 lines (ORPHAN)
"Article N Next Article"	5 lines (ORPHAN - no asterisks)
"Article N" (bare)	2 lines (ORPHAN)

The pipe-separated variant was the big one. It looks like HTML-export artifacts from Factiva/LexisNexis where the "Previous Article" navigation link was separated from the asterisk line by a pipe character.

Updated SEP_NEWSPAPER (create_excelV5.py line 45):

```
SEP_NEWSPAPER = re.compile(
    r"(?:"
    r"^\\s*Article\\s+\\d+"
    r"(?:(?:\\s+(?:Previous|Next)\\s+Article)?\\s*\\|?\\s*\\*{10,})?"
    r"\\s*$"
    r"|^\\s*\\*\\s+\\*\\s+\\*\\s*$"
    r")",
    re.MULTILINE | re.IGNORECASE,
)
```



Validation performed before applying the fix:

- Zero old matches dropped -- existing 3373 boundaries still hit.
- 95 new matches added -- all confirmed to be on lines starting with "Article N".
- Min chunk size post-fix: 233 chars. No micro-chunks indicating over-splitting.

- No false positives: no "Article N <body text>" matches, because \$ anchor requires end-of-line.

Impact:

- Newspapers final count: 3354 -> 3449 (+95 articles)
- Newspapers coverage: 89.2% -> 91.7%
- Overall output.xlsx: 12,111 -> 12,206 rows
- Overall coverage: 94.8% -> 95.6%

Next steps

Remaining concrete follow-ups:

- Science News -172 and Newspapers -313 are articles absent from the raw text files -- inspect Science_news.txt and Newspapers_1984-2005.txt against the reference worksheet to confirm which specific articles are missing, then re-scrape from the original source. Not a parser fix.
- Futurists is over-splitting (raw=1065 vs ref=926). Its combined_sep in parse_futurist() matches three separators, including a dash-run "-----" that may be firing inside article bodies used for formatting. The overshoot is currently absorbed by boilerplate and dedup filters -- which means dates are aligned approximately, not exactly. Worth tightening.
- Calibrate the qualitative-coding output: hand-code 10 articles and compare against Claude's output. Target $\geq 80\%$ agreement on Attitude and Sustaining/Disrupting (the more subjective dimensions). If lower, add more few-shots to qualitative_coding/prompts.py.
- Consider widening the tolerant metric's decay window from 90 days to 180 days if the 92.6% tolerant score for Science News proves to penalize too many legitimate same-year-different-month matches. Adjustable in create_excelV5.py:_compute_date_match_tolerant.

Deferred:

- Community dimension graphs over time (next_steps.txt bullet 2, marked IGNORE). The Claude coding module produces exactly the per-article codings needed to drive these graphs -- once it has been run, a new notebook section can read the coded sheets and plot the requested series.